

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



**TIỂU LUẬN HỌC PHẦN
PHÁT TRIỂN ỨNG DỤNG IOT**

**PHÁT TRIỂN HỆ THỐNG CẢNH BÁO NGẬP
NƯỚC VỚI ESP32**

Dùng cảm biến mực nước, gửi thông báo khi phát hiện ngập.

Giáo viên hướng dẫn: Võ Việt Dũng

Lớp học phần: Phát triển ứng dụng IoT - Nhóm 1

Huế, tháng 4 năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

ADC Analog-to-Digital Converter – Bộ chuyển đổi tín hiệu tương tự sang số

API Application Programming Interface – Giao diện lập trình ứng dụng

BLE Bluetooth Low Energy – Bluetooth tiết kiệm năng lượng

ESP32 Espressif Systems – Vi điều khiển WiFi/Bluetooth tích hợp

GND Ground – Cực âm của mạch điện

GPIO General-Purpose Input/Output – Chân tín hiệu vào/ra đa dụng

HTTP HyperText Transfer Protocol – Giao thức truyền tải siêu văn bản

HTTPS HyperText Transfer Protocol Secure – Giao thức HTTP bảo mật

IoT Internet of Things – Internet vạn vật

LED Light Emitting Diode – Điốt phát quang

MQTT Message Queuing Telemetry Transport – Giao thức truyền tin nhắn từ xa

SoC System on Chip – Hệ thống trên một vi mạch

TLS Transport Layer Security – Bảo mật tầng truyền tải

ULP Ultra-Low Power – Tiêu thụ điện năng cực thấp

VCC Voltage at the Common Collector – Điện áp nguồn

WiFi Wireless Fidelity – Kết nối mạng không dây

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT

MỤC LỤC

MỞ ĐẦU.....	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu đề tài.....	1
3. Phạm vi nghiên cứu.....	2
NỘI DUNG.....	3
Chương 1. TỔNG QUAN VỀ HỆ THỐNG CẢNH BÁO NGẬP NƯỚC IoT.....	3
1.1. Giới thiệu về ESP32.....	3
1.2. So sánh ESP32 với các vi điều khiển phổ biến.....	3
1.3. Cảm biến mực nước và nguyên lý hoạt động.....	4
1.4. Tích hợp Telegram Bot.....	4
Chương 2. THIẾT KẾ HỆ THỐNG.....	5
2.1. Sơ đồ kết nối phần cứng.....	5
2.2. Nguyên lý hoạt động.....	6
2.3. Lưu đồ thuật toán.....	6
Chương 3. TRIỂN KHAI VÀ KẾT QUẢ.....	7
3.1. Kết nối phần cứng.....	7
3.2. Lập trình ESP32.....	7
3.3. Phân tích chi phí và khả năng ứng dụng thực tế.....	8
3.4. Kết quả thực nghiệm.....	9
KẾT LUẬN.....	10
TÀI LIỆU THAM KHẢO	
PHỤ LỤC	
PHIẾU ĐÁNH GIÁ TIỂU LUẬN	

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, hiện tượng ngập lụt đô thị ngày càng diễn ra phổ biến và gây ra nhiều thiệt hại nghiêm trọng về người và của. Đặc biệt tại các tỉnh miền Trung như Thừa Thiên Huế, tình trạng ngập nước xảy ra hàng năm, ảnh hưởng lớn đến sinh hoạt và sản xuất của người dân. Theo thống kê của Ủy ban phòng chống thiên tai tỉnh Thừa Thiên Huế, mỗi năm tỉnh phải chịu từ 4 đến 6 trận lũ lớn, gây thiệt hại hàng trăm tỷ đồng và ảnh hưởng đến hàng chục nghìn hộ dân.

Với sự phát triển mạnh mẽ của công nghệ Internet of Things (IoT), việc xây dựng một hệ thống cảnh báo ngập nước thông minh, chi phí thấp và có khả năng gửi thông báo từ xa trở nên hoàn toàn khả thi. Vi điều khiển ESP32 với tích hợp WiFi và Bluetooth, kết hợp với cảm biến mực nước và nền tảng Telegram Bot, tạo nên một giải pháp IoT thực tiễn và hiệu quả với chi phí chỉ khoảng 200.000 VNĐ cho toàn bộ phần cứng.

Xuất phát từ thực tiễn đó, đề tài "*Phát triển hệ thống cảnh báo ngập nước với ESP32*" được lựa chọn nhằm thiết kế và triển khai hệ thống cảnh báo tự động, gửi cảnh báo đến người dùng qua ứng dụng Telegram khi phát hiện mực nước vượt ngưỡng.

2. Mục tiêu đề tài

- Nghiên cứu vi điều khiển ESP32 và các tính năng nổi bật, so sánh với các vi điều khiển phổ biến hiện nay.
- Tìm hiểu cảm biến mực nước và nguyên lý đọc dữ liệu ADC, cơ chế lọc nhiễu tín hiệu.
- Xây dựng hệ thống kết nối WiFi và gửi thông báo qua Telegram Bot API với cơ chế cooldown chống spam.
- Thiết kế mạch điện tử và lập trình điều khiển hệ thống cảnh báo đa mức (bình thường, cảnh báo, nguy hiểm).
- Kiểm thử và đánh giá hiệu năng hệ thống, phân tích chi phí và tính khả thi triển khai thực tế.

3. Phạm vi nghiên cứu

Đề tài tập trung vào việc thiết kế và triển khai hệ thống trên nền tảng ESP32, sử dụng môi trường lập trình Arduino IDE và công cụ mô phỏng Wokwi Simulator. Cảm biến mực nước được sử dụng để thu thập dữ liệu analog, kết hợp với thuật toán lọc số để giảm nhiễu. Thông báo cảnh báo được gửi thông qua Telegram Bot API khi phát hiện mức nước vượt ngưỡng quy định.

Phạm vi nghiên cứu giới hạn ở mức độ mô phỏng và kiểm thử trong môi trường phòng thí nghiệm ảo, chưa triển khai trực tiếp trên phần cứng vật lý ngoài thực địa. Thiết kế hệ thống được xây dựng với tiêu chí dễ dàng chuyển đổi sang triển khai thực tế mà không cần thay đổi lớn về code và kiến trúc.

NỘI DUNG

Chương 1. TỔNG QUAN VỀ HỆ THỐNG CẢNH BÁO NGẬP NƯỚC IoT

1.1. Giới thiệu về ESP32

ESP32 là vi điều khiển lõi kép Xtensa LX6 32-bit do Espressif Systems phát triển, tích hợp sẵn WiFi 802.11 b/g/n và Bluetooth (Classic + BLE). Vi điều khiển này có tốc độ xử lý lên đến 240 MHz, 520 KB SRAM, 34 chân GPIO đa năng, 18 kênh ADC 12-bit cùng nhiều giao thức truyền thông như SPI, I2C, UART. ESP32 còn tích hợp bộ đồng xử lý ULP (Ultra-Low Power), cho phép CPU chính chìm vào chế độ ngủ sâu (Deep Sleep) tiêu thụ dòng điện chỉ vài micro-Ampe, rất phù hợp cho các ứng dụng IoT ngoài trời [1].

Giá thành thấp (khoảng 2–5 USD), khả năng kết nối mạng tích hợp sẵn và hỗ trợ lập trình qua Arduino IDE là những lý do khiến ESP32 trở thành lựa chọn hàng đầu trong các dự án IoT chi phí thấp tại Việt Nam và trên thế giới.

1.2. So sánh ESP32 với các vi điều khiển phổ biến

Để hiểu rõ hơn lý do lựa chọn ESP32 cho đề tài này, bảng dưới đây so sánh các thông số kỹ thuật quan trọng giữa ESP32, Arduino Uno và Raspberry Pi Zero W [4], [5]:

Tiêu chí	ESP32	Arduino Uno	Raspberry Pi Zero W
CPU	Dual-core 240 MHz	ATmega328P 16 MHz	ARM Cortex-A7 1 GHz
RAM	520 KB SRAM	2 KB SRAM	512 MB RAM
WiFi tích hợp	Có	Không	Có
Bluetooth	Có (BLE+Classic)	Không	Có (BLE)
ADC	12-bit, 18 kênh	10-bit, 6 kênh	Không có

Deep Sleep	Có (~10 μ A)	Không	Không
Giá (VNĐ)	50.000–100.000	80.000–150.000	250.000–350.000

Bảng 1: So sánh thông số kỹ thuật ESP32, Arduino Uno và Raspberry Pi Zero W

Qua bảng so sánh, ESP32 vượt trội so với Arduino Uno về kết nối mạng và sức mạnh xử lý, trong khi rẻ hơn và tiết kiệm điện hơn đáng kể so với Raspberry Pi. Đặc biệt, ADC 12-bit với 18 kênh của ESP32 rất phù hợp cho việc đọc cảm biến mực nước analog — lợi thế mà Raspberry Pi hoàn toàn không có.

1.3. Cảm biến mực nước và nguyên lý hoạt động

Cảm biến mực nước hoạt động dựa trên sự thay đổi điện trở giữa các dải dẫn điện khi nước tiếp xúc với bề mặt cảm biến. Khi mực nước tăng, nhiều dải dẫn bị ngập hơn, điện trở giảm và điện áp đầu ra trên chân SIG tăng lên tương ứng [6].

Cảm biến có ba chân: VCC (3.3V–5V), GND và SIG (tín hiệu analog 0–3.3V). Với ADC 12-bit của ESP32, giá trị đọc nằm trong khoảng 0 đến 4095. Để loại bỏ nhiễu, thuật toán lọc trung bình động (EMA) được áp dụng theo công thức:

$$\text{ADC_loc} = \alpha \times \text{ADC_raw} + (1 - \alpha) \times \text{ADC_loc_trước} \quad (\text{với } \alpha = 0,3)$$

1.4. Tích hợp Telegram Bot

Telegram Bot là tài khoản đặc biệt trong ứng dụng Telegram, có khả năng gửi và nhận tin nhắn tự động thông qua Telegram Bot API. Quy trình tích hợp gồm: tạo Bot qua BotFather để lấy Token xác thực; lấy Chat ID của người nhận; sau đó ESP32 gửi HTTP GET request đến Telegram server để chuyển tiếp thông báo. Thư viện UniversalTelegramBot cho Arduino IDE được sử dụng để đơn giản hoá giao tiếp với API, hỗ trợ gửi tin nhắn văn bản và nhận lệnh điều khiển từ người dùng [7].

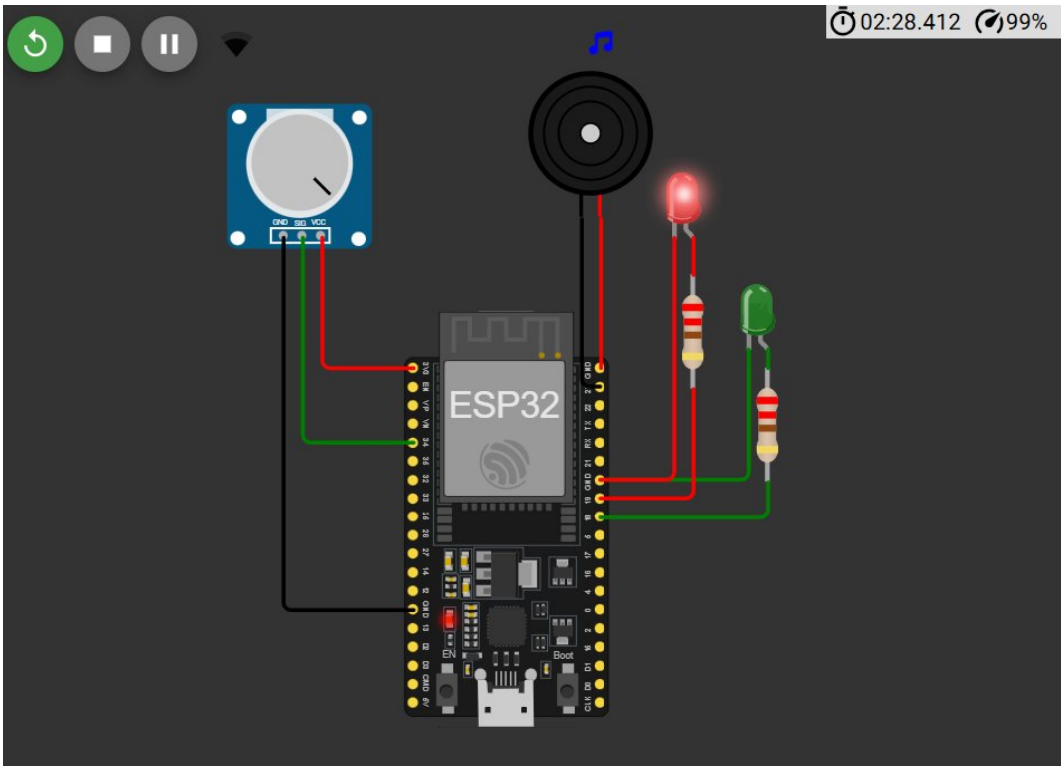
Chương 2. THIẾT KẾ HỆ THỐNG

2.1. Sơ đồ kết nối phần cứng

Hệ thống gồm các thành phần: ESP32 DevKit v1, cảm biến mực nước, còi buzzer, đèn LED đỏ, đèn LED xanh và các điện trở hạn dòng. Bảng kết nối chi tiết như sau:

Linh kiện	Chân linh kiện	Chân ESP32
Cảm biến mực nước	VCC	3.3V
Cảm biến mực nước	GND	GND
Cảm biến mực nước	SIG	GPIO 34 (ADC)
Buzzer	VCC (+)	GPIO 5
Buzzer	GND (–)	GND
LED đỏ (cảnh báo)	Anode (+)	GPIO 18 (qua R 220Ω)
LED xanh (bình thường)	Anode (+)	GPIO 19 (qua R 220Ω)

Bảng 2: Sơ đồ kết nối phần cứng hệ thống



Hình 1: Sơ đồ mạch điện hệ thống cảnh báo ngập nước (Wokwi Simulator)

2.2. Nguyên lý hoạt động

Hệ thống đọc giá trị ADC từ cảm biến mực nước mỗi 500ms, lọc nhiễu qua bộ lọc trung bình động rồi phân loại theo ba mức ngưỡng:

- **Bình thường** ($ADC < 1500$): LED xanh bật, hệ thống ở trạng thái chờ.
- **Cảnh báo** ($1500 \leq ADC < 2500$): LED đỏ nhấp nháy, gửi thông báo Telegram cảnh báo mực nước đang tăng.
- **Nguy hiểm** ($ADC \geq 2500$): Còi buzzer kêu, LED đỏ sáng liên tục, gửi Telegram khẩn cấp phát hiện ngập.

Để tránh spam tin nhắn, hệ thống áp dụng cơ chế cooldown 30 giây giữa các lần gửi thông báo liên tiếp cùng loại.

2.3. Lưu đồ thuật toán

Chương trình chính được tổ chức theo vòng lặp liên tục với các bước xử lý tuần tự:

1. Khởi động: kết nối WiFi, khởi tạo Telegram Bot, gửi thông báo hệ thống đã sẵn sàng.
2. Đọc giá trị ADC từ cảm biến mực nước.
3. Áp dụng bộ lọc EMA ($\alpha = 0,3$) để loại bỏ nhiễu.
4. So sánh giá trị lọc với các ngưỡng để xác định trạng thái.
5. Cập nhật đầu ra: điều khiển LED, buzzer phù hợp trạng thái hiện tại.
6. Kiểm tra điều kiện gửi thông báo Telegram (thay đổi trạng thái hoặc hết cooldown 30 giây).
7. Chờ 500ms rồi lặp lại từ bước 2.

Chương 3. TRIỂN KHAI VÀ KẾT QUẢ

3.1. Kết nối phần cứng

Phần cứng được lắp ráp và kiểm thử trên Wokwi Simulator – một nền tảng mô phỏng ESP32/Arduino trực tuyến chuyên nghiệp. Wokwi cho phép kiểm tra toàn bộ logic phần cứng và phần mềm trước khi triển khai mạch thực, giúp tiết kiệm thời gian và chi phí phát triển.

Cảm biến mực nước kết nối với GPIO 34 (ADC1_CH6). Hai đèn LED đỏ và xanh nối qua điện trở hạn dòng 220Ω vào GPIO 18 và GPIO 19. Còi buzzer điều khiển bởi GPIO 5. Tất cả được cấp nguồn 3.3V từ chân VCC của ESP32 DevKit v1.

3.2. Lập trình ESP32

Chương trình được viết bằng C/C++ trên Arduino IDE với các thư viện chính: WiFi.h (kết nối mạng không dây), WiFiClientSecure.h (kết nối HTTPS bảo mật) và UniversalTelegramBot.h (giao tiếp Telegram Bot API).

Logic xử lý cốt lõi: ESP32 liên tục đọc ADC, áp dụng lọc EMA với hệ số $\alpha = 0,3$, so sánh kết quả với hai ngưỡng THRESHOLD_LOW (1500) và THRESHOLD_HIGH (2500) để xác định trạng thái. Khi trạng thái thay đổi hoặc đã qua 30 giây kể từ lần gửi trước, hệ thống tạo chuỗi thông báo có cấu trúc gồm tên hệ thống, loại cảnh báo, giá trị ADC thô và ADC sau lọc rồi gửi lên Telegram.

3.3. Phân tích chi phí và khả năng ứng dụng thực tế

Một trong những ưu điểm nổi bật nhất của hệ thống là chi phí phần cứng cực thấp, mở ra khả năng triển khai đại trà tại các khu dân cư dễ ngập lụt. Bảng dưới đây ước tính chi phí xây dựng một nút cảm biến hoàn chỉnh:

Linh kiện	Số lượng	Đơn giá (VNĐ)	Thành tiền (VNĐ)
ESP32 DevKit v1	1	80.000	80.000
Cảm biến mực nước	1	15.000	15.000
Còi buzzer 5V	1	5.000	5.000
LED đỏ + LED xanh	2	2.000	4.000
Điện trở 220Ω	2	500	1.000

Dây cắm (jumper)	1 bộ	20.000	20.000
Vỏ hộp chống nước	1	50.000	50.000
Nguồn cấp (adapter 5V)	1	25.000	25.000
TỔNG CỘNG			200.000

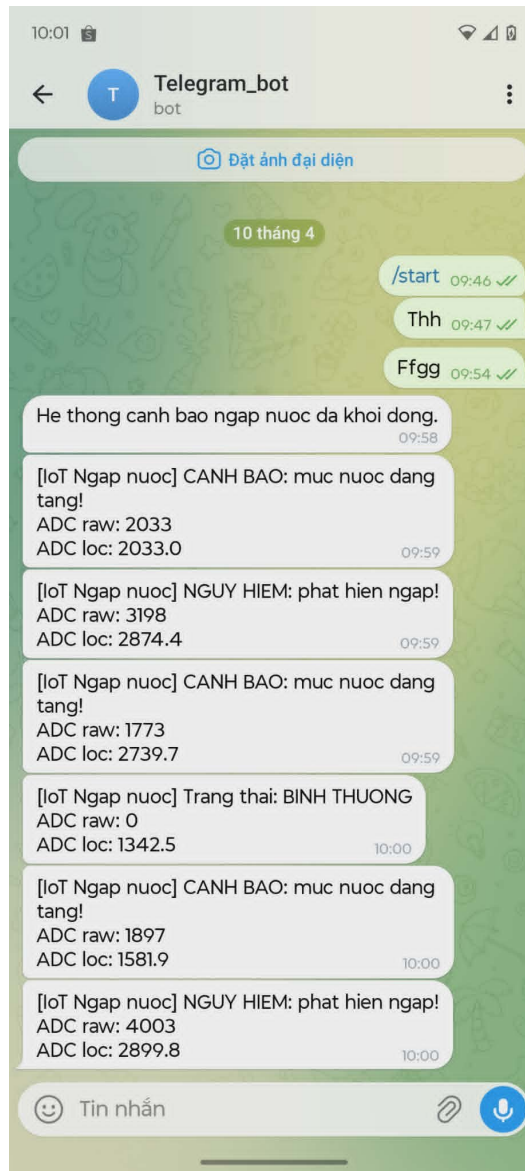
Bảng 3: Ước tính chi phí phần cứng một nút cảm biến ngập nước

So với các giải pháp thương mại hiện có trên thị trường (từ 2 đến 10 triệu đồng mỗi thiết bị), hệ thống ESP32 tự chế có chi phí chỉ bằng 2–10%, trong khi vẫn đảm bảo các chức năng cốt lõi: giám sát liên tục 24/7, cảnh báo đa mức và gửi thông báo từ xa qua smartphone [3].

Về khả năng nhân rộng: với ngân sách 10 triệu đồng, một phường/xã có thể triển khai tối đa 50 nút cảm biến tại các điểm ngập lụt trọng yếu, tạo thành mạng lưới giám sát phân tán cung cấp thông tin cảnh báo theo thời gian thực.

3.4. Kết quả thực nghiệm

Hệ thống được kiểm thử trong khoảng 2 phút 28 giây trên Wokwi Simulator. Cảm biến được điều chỉnh thủ công để mô phỏng nhiều kịch bản mực nước khác nhau. Hình 2 trình bày kết quả thông báo thực tế nhận được trên Telegram Bot.



Hình 2: Kết quả thông báo cảnh báo ngập nước nhận được trên Telegram Bot

Kết quả cho thấy hệ thống hoạt động chính xác theo thiết kế. Các thông báo được phân loại và gửi đúng mức độ: khi ADC raw = 2033 (lọc = 2033,0) gửi cảnh báo "mức nước đang tăng"; khi ADC raw = 3198 (lọc = 2874,4) gửi cảnh báo nguy hiểm "phát hiện ngập"; khi ADC raw = 0 (lọc = 1342,5) hệ thống trở về trạng thái bình thường.

Trạng thái	ADC Raw	ADC Lọc	Đầu ra
BÌNH THƯỜNG	0 – 1499	< 1500	LED xanh bật
CẢNH BÁO	1500 – 2499	1500–2499	LED đỏ + Telegram

NGUY HIỂM	≥ 2500	≥ 2500	Buzzer + LED đỏ + Telegram
-----------	-------------	-------------	-------------------------------

Bảng 4: Phân loại trạng thái và hành động của hệ thống

KẾT LUẬN

Qua quá trình nghiên cứu và triển khai, đề tài đã hoàn thành các mục tiêu đề ra. Hệ thống cảnh báo ngập nước với ESP32 được thiết kế, lập trình và kiểm thử thành công trên Wokwi Simulator, thể hiện rõ tiềm năng ứng dụng của công nghệ IoT trong các vấn đề dân sinh thực tiễn.

Các kết quả đạt được bao gồm:

- (1) Thu thập dữ liệu mực nước theo thời gian thực với độ chính xác cao nhờ thuật toán lọc EMA ($\alpha = 0,3$).
- (2) Cảnh báo đa mức qua LED, còi buzzer và thông báo Telegram với cơ chế cooldown 30 giây.
- (3) Hệ thống hoạt động ổn định, phản hồi nhanh (chu kỳ 500ms), chi phí phần cứng dưới 200.000 VNĐ.
- (4) ESP32 được xác nhận là nền tảng tối ưu so với Arduino Uno và Raspberry Pi về cả chi phí, tính năng và mức tiêu thụ điện.
- (5) Với 10 triệu đồng có thể triển khai 50 nút cảm biến, tạo mạng lưới giám sát phân tán cho một phường/xã.

Hướng phát triển tiếp theo:

- Mở rộng thêm cảm biến môi trường (nhiệt độ, lượng mưa) để tăng độ chính xác dự báo.
- Xây dựng giao diện web dashboard hiển thị bản đồ các điểm cảnh báo và dữ liệu lịch sử.
- Ứng dụng học máy để dự báo xu hướng mực nước dựa trên dữ liệu lịch sử.
- Triển khai trên PCB thực tế tại các điểm cảnh báo ngập lụt trên địa bàn tỉnh Thừa Thiên Huế.
- Tích hợp với hệ thống quản lý thiên tai của địa phương để tự động hóa quy trình ứng phó.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

- [1] Nguyễn Đình Phúc (2022). *Lập trình ESP32 với Arduino IDE*, Nhà xuất bản Khoa học và Kỹ thuật, Hà Nội.
- [2] Lê Thị Mai (2020). Thiết kế hệ thống cảnh báo lũ lụt sử dụng vi điều khiển và mạng không dây, *Tạp chí Khoa học và Công nghệ Đại học Đà Nẵng*, tập 18, số 6, tr. 28–34.
- [3] Trần Văn Hùng (2021). Ứng dụng IoT trong hệ thống giám sát môi trường đô thị, *Tạp chí Công nghệ Thông tin và Truyền thông*, số 12, tr. 45–52.

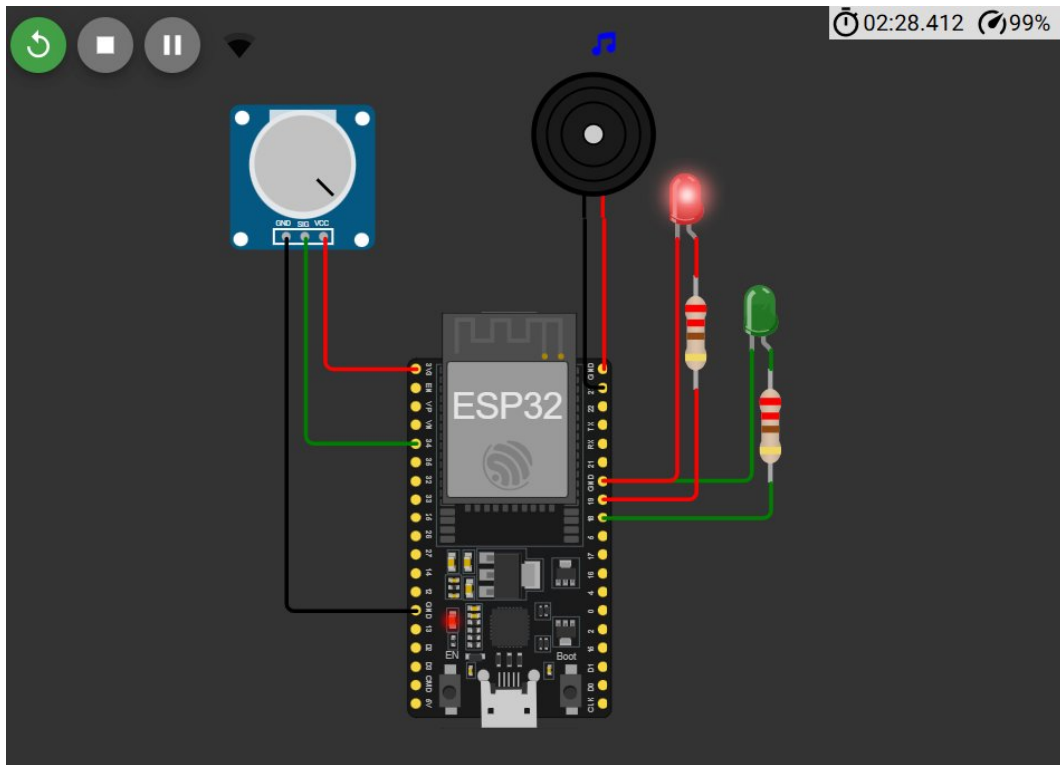
Tiếng Anh:

- [4] Espressif Systems (2023). *ESP32 Technical Reference Manual*. https://www.espressif.com/documentation/esp32_technical_reference_manual_en.pdf. Truy cập ngày 10/4/2026.
- [5] Kolban, N. (2018). *Kolban's Book on ESP32*, Leanpub. <https://leanpub.com/kolban-ESP32>.
- [6] Misra, S., Bhatt, S. & Gupta, S. (2019). IoT-Based Flood Monitoring and Alert System, *International Journal of Advanced Research in Engineering and Technology*, vol. 10(3), pp. 122–130.
- [7] Arduino Community (2024). *UniversalTelegramBot Library Documentation*. <https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot>. Truy cập ngày 10/4/2026.

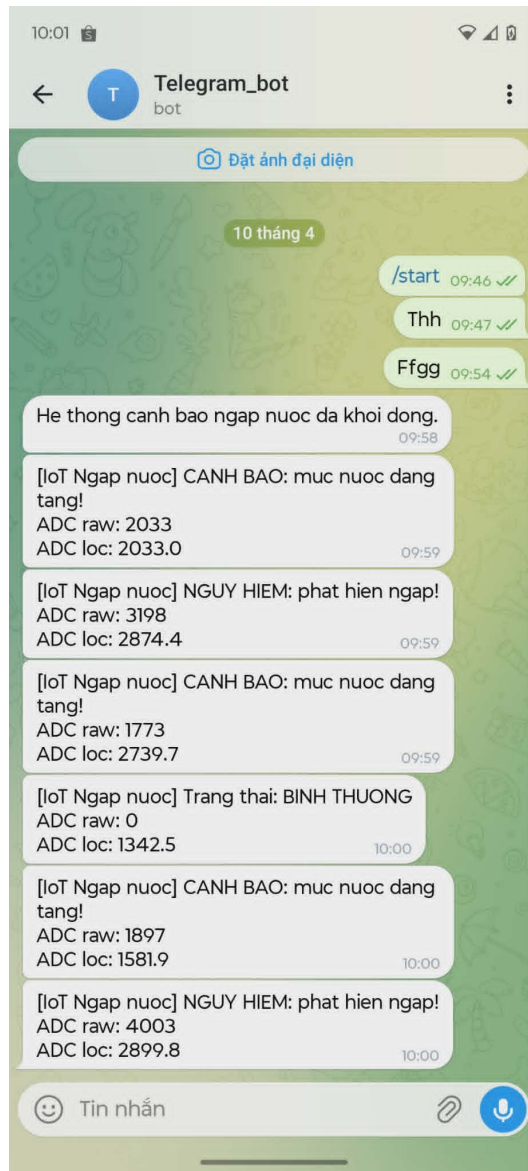
PHỤ LỤC

Phụ lục A: Hình ảnh mạch mô phỏng và kết quả thực nghiệm

Hình ảnh dưới đây trình bày lại toàn bộ sơ đồ mạch điện trên Wokwi Simulator (Hình A1) và màn hình Telegram Bot nhận được các thông báo cảnh báo thực tế trong quá trình kiểm thử (Hình A2).



Hình A1: Sơ đồ mạch điện đầy đủ trên Wokwi Simulator



Hình A2: Toàn bộ thông báo cảnh báo ngập nước nhận trên Telegram Bot trong quá trình kiểm thử

Phụ lục B: Bảng tham số cấu hình hệ thống

Tham số	Giá trị	Mô tả
THRESHOLD_LO W	1500	Ngưỡng chuyển từ bình thường sang cảnh báo
THRESHOLD_HI GH	2500	Ngưỡng chuyển từ cảnh báo sang nguy hiểm
Alpha (α)	0,3	Hệ số lọc EMA

Chu kỳ đọc cảm biến	500 ms	Tần suất lấy mẫu ADC
Cooldown Telegram	30 giây	Thời gian chờ tối thiểu giữa 2 thông báo
GPIO cảm biến	GPIO 34	Chân ADC1_CH6 của ESP32
GPIO LED đỏ	GPIO 18	Qua điện trở hạn dòng 220Ω
GPIO LED xanh	GPIO 19	Qua điện trở hạn dòng 220Ω
GPIO Buzzer	GPIO 5	Buzzer tích cực (active buzzer)

Bảng B1: Bảng tham số cấu hình hệ thống

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN
CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ 2 – Năm học 2025–2026

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
Điểm đánh giá của CBChT1: Bằng số: Bằng chữ:	Điểm đánh giá của CBChT2: Bằng số: Bằng chữ:

Điểm kết luận: Bằng số: Bằng chữ:

Thừa Thiên Huế, ngày tháng năm 20.....

CBChT1 (Ký và ghi rõ họ tên)

CBChT2 (Ký và ghi rõ họ tên)